Python Lab

Assignment Collection

Assignments 1 – 10 • Python Programming Lab

10	60+	Python 3
Assignments	Programs	Language

Assignment 1

Q1. Swap Two Numbers Without a Third Variable

◆ PYTHON CODE

```
a = int(input("Enter 1st no: "))
b = int(input("Enter 2nd no: "))

a = a + b
b = a - b
a = a - b

print("After swap a =", a, "b =", b)
```

● OUTPUT

```
Enter 1st no: 5
Enter 2nd no: 8
After swap a = 8 b = 5
```

Q2. Basic Arithmetic Operations (Add, Subtract, Multiply, Divide)

◆ PYTHON CODE

```
a = int(input("Enter the 1st no: "))
b = int(input("Enter the 2nd no: "))

print("Addition = ", a + b)
print("Subtraction = ", a - b)
print("Multiplication = ", a * b)
print("Divide = ", a / b)
```

● OUTPUT

```
Enter the 1st no: 10
Enter the 2nd no: 2
Addition = 12
Subtraction = 8
Multiplication = 20
Divide = 5.0
```

Q3. Calculate Percentage Scored

◆ PYTHON CODE

```
Total = float(input("Enter total marks: "))
obt = float(input("Enter marks obtained: "))

per = (obt / Total) * 100
print("Percentage = ", per, "%")
```

● OUTPUT

```
Enter total marks: 500
Enter marks obtained: 450
Percentage = 90.0 %
```

Q4. Convert Temperature from Centigrade to Fahrenheit

◆ PYTHON CODE

```
celsius = float(input("Enter temperature in Centigrade: "))
fahrenheit = (celsius * 9/5) + 32

print("Temperature in Fahrenheit = ", fahrenheit)
```

● OUTPUT

```
Enter temperature in Centigrade: 37
Temperature in Fahrenheit = 98.6
```

Q5. Convert Total Days into Years, Weeks, and Days

◆ PYTHON CODE

```
total_days = int(input("Enter total days: "))

years = total_days // 365
remaining_days = total_days % 365
weeks = remaining_days // 7
days = remaining_days % 7

print("Years =", years)
print("Weeks =", weeks)
print("Days =", days)
```

● OUTPUT

```
Enter total days: 397
Years = 1
Weeks = 4
Days = 4
```

Q6. Calculate Simple and Compound Interest

◆ PYTHON CODE

```
p = float(input("Enter Principle amount: "))
r = float(input("Enter Rate of interest (%): "))
t = float(input("Enter Time (in years): "))

# Simple Interest
si = (p * r * t) / 100

# Compound Interest
ci = p * ((1 + r / 100) ** t) - p

print("Simple Interest = ", round(si, 2))
print("Compound Interest = ", round(ci, 2))
```

● OUTPUT

```
Enter Principle amount: 10000
Enter Rate of interest (%): 5
Enter Time (in years): 2
Simple Interest = 1000.0
Compound Interest = 1025.0
```

Q7. Bitwise Operations

◆ PYTHON CODE

```
a = int(input("Enter 1st no: "))
b = int(input("Enter 2nd no: "))

print("Bitwise AND (a & b) =", a & b)
print("Bitwise OR (a | b) =", a | b)
print("Bitwise XOR (a ^ b) =", a ^ b)
print("Bitwise NOT (~a) =", ~a)
print("Left Shift (a << 2) =", a << 2)
print("Right Shift (a >> 2) =", a >> 2)
```

● OUTPUT

```
Enter 1st no: 5
Enter 2nd no: 3
Bitwise AND (a & b) = 1
Bitwise OR (a | b) = 7
Bitwise XOR (a ^ b) = 6
Bitwise NOT (~a) = -6
Left Shift (a << 2) = 20
Right Shift (a >> 2) = 1
```

Q8. Math Functions (Square Root, Floor, Ceil, etc.)

◆ PYTHON CODE

```
import math

num1 = float(input("Enter a decimal number: "))
num2 = float(input("Enter a number for square root: "))

print("Square root =", math.sqrt(num2))
print("Floor value =", math.floor(num1))
print("Ceil value =", math.ceil(num1))
print("Absolute value =", math.fabs(-num1))
print("Power (2^3) =", math.pow(2, 3))
```

● OUTPUT

```
Enter a decimal number: 5.7
Enter a number for square root: 25
Square root = 5.0
Floor value = 5
Ceil value = 6
Absolute value = 5.7
Power (2^3) = 8.0
```

Assignment 2

Q1. Area and Circumference of a Circle

◆ PYTHON CODE

```
import math

radius = float(input("Enter radius: "))
area = math.pi * radius * radius
circumference = 2 * math.pi * radius

print("Area = ", round(area, 2))
print("Circumference = ", round(circumference, 2))
```

● OUTPUT

```
Enter radius: 5
Area = 78.54
Circumference = 31.42
```

Q2. Area of a Triangle

◆ PYTHON CODE

```
base = float(input("Enter base: "))
height = float(input("Enter height: "))
area = 0.5 * base * height

print("Area of Triangle = ", area)
```

● OUTPUT

```
Enter base: 10
Enter height: 5
Area of Triangle = 25.0
```

Q3. Convert Centimeter to Meter and Kilometer

◆ PYTHON CODE

```
cm = float(input("Enter length in cm: "))
meter = cm / 100
km = cm / 100000

print("Meter = ", meter)
print("Kilometer = ", km)
```

● OUTPUT

```
Enter length in cm: 500000
Meter = 5000.0
Kilometer = 5.0
```

Q4. Relational Operators

◆ PYTHON CODE

```
a = int(input("Enter 1st no: "))
b = int(input("Enter 2nd no: "))

print(a, ">", b, "is", a > b)
print(a, "<", b, "is", a < b)
print(a, "==", b, "is", a == b)
print(a, "!=", b, "is", a != b)
```

● OUTPUT

```
Enter 1st no: 15
Enter 2nd no: 10
15 > 10 is True
15 < 10 is False
15 == 10 is False
15 != 10 is True
```

Q5. Assignment Operators

◆ PYTHON CODE

```
x = int(input("Enter a number: "))

# Simple assignment
print("Initial x =", x)

# Add and assign
x += 5
print("After x += 5 ->", x)

# Subtract and assign
x -= 3
print("After x -= 3 ->", x)

# Multiply and assign
x *= 2
print("After x *= 2 ->", x)
```

● OUTPUT

```
Enter a number: 10
Initial x = 10
After x += 5 -> 15
After x -= 3 -> 12
After x *= 2 -> 24
```

Q6. Max and Min Among 3 Elements Using Ternary Operator

◆ PYTHON CODE

```
a = int(input("Enter 1st no: "))
b = int(input("Enter 2nd no: "))
c = int(input("Enter 3rd no: "))

# Ternary operator syntax: [true_value] if [condition] else [false_value]
maximum = a if (a > b and a > c) else (b if b > c else c)
minimum = a if (a < b and a < c) else (b if b < c else c)

print("Maximum element =", maximum)
print("Minimum element =", minimum)
```

● OUTPUT

```
Enter 1st no: 25
Enter 2nd no: 12
Enter 3rd no: 48
Maximum element = 48
Minimum element = 12
```

Q7. Compare Two Numbers Using Ternary Operator

◆ PYTHON CODE

```
num1 = int(input("Enter 1st no: "))
num2 = int(input("Enter 2nd no: "))

result = "Both are equal" if num1 == num2 else ("1st no is greater" if num1 > num2 else "2nd
no is greater")

print("Result =", result)
```

● OUTPUT

```
Enter 1st no: 40
Enter 2nd no: 55
Result = 2nd no is greater
```

Q8. Add Number and Numeric String Using Explicit Type Conversion

◆ PYTHON CODE

```
num = int(input("Enter a number: "))
num_str = input("Enter a numeric string: ")

# Explicitly converting string to integer
result = num + int(num_str)

print("Result of addition =", result)
```

● OUTPUT

```
Enter a number: 50
Enter a numeric string: 100
Result of addition = 150
```

Assignment 3

Q1. Check if Temperature is Freezing

◆ PYTHON CODE

```
temp = float(input("Enter temperature in Celsius: "))

if temp <= 0:
print("Temperature is Freezing.")
else:
print("Temperature is Not Freezing.")
```

● OUTPUT

```
Enter temperature in Celsius: -2.5
Temperature is Freezing.
```

Q2. Division Based on Marks

◆ PYTHON CODE

```
marks = int(input("Enter marks: "))

if marks >= 600:
print("Result: 1st Division")
elif marks >= 450 and marks <= 599:
print("Result: 2nd Division")
elif marks >= 300 and marks <= 449:
print("Result: 3rd Division")
else:
print("Result: Fail")
```

● OUTPUT

```
Enter marks: 520
Result: 2nd Division
```

Q3. Check Character Type (Alphabet, Digit, or Special Character)

◆ PYTHON CODE

```
ch = input("Enter any character: ")

if ch.isalpha():
print(ch, "is an Alphabet.")
elif ch.isdigit():
print(ch, "is a Digit.")
else:
print(ch, "is a Special Character.")
```

● OUTPUT

```
Enter any character: @
@ is a Special Character.
```

Q4. Check Odd or Even Using Bitwise Operator and Conditional Statement

◆ PYTHON CODE

```
num = int(input("Enter a number: "))

# Using bitwise AND (&).
# If a number is odd, its binary form ends in 1, so num & 1 equals 1.
if (num & 1) == 1:
print(num, "is Odd.")
else:
print(num, "is Even.")
```

● OUTPUT

```
Enter a number: 17
17 is Odd.
```

Q5. Choice-Based Calculator

◆ PYTHON CODE

```
print("--- Choice Based Calculator ---")
print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")

choice = int(input("Enter your choice (1-4): "))

a = float(input("Enter 1st no: "))
b = float(input("Enter 2nd no: "))

if choice == 1:
print("Result = ", a + b)
elif choice == 2:
print("Result = ", a - b)
elif choice == 3:
print("Result = ", a * b)
elif choice == 4:
if b != 0:
print("Result = ", a / b)
else:
print("Error: Division by zero is not allowed.")
else:
print("Invalid Choice!")
```

● OUTPUT

```
--- Choice Based Calculator ---
```

Q1. Addition

Q2. Subtraction

Q3. Multiplication

Q4. Division

Assignment 4

Q1. Calculate Electricity Bill Amount with Tax

◆ PYTHON CODE

```
bill_amt = float(input("Enter the initial bill amount: "))

if bill_amt < 800:
tax = 0.0
elif bill_amt >= 800 and bill_amt < 1200:
tax = 0.10 * bill_amt
elif bill_amt >= 1200 and bill_amt < 2000:
tax = 0.15 * bill_amt
else:
tax = 0.22 * bill_amt

total_bill = bill_amt + tax

print("Tax Amount = ", round(tax, 2))
print("Total Bill Amount (with tax) = ", round(total_bill, 2))
```

● OUTPUT

```
Enter the initial bill amount: 1500
Tax Amount = 225.0
Total Bill Amount (with tax) = 1725.0
```

Q2. Count Currency Denominations

◆ PYTHON CODE

```
amount = int(input("Enter total amount of money: "))

# Tuple containing available note denominations
denominations = (2000, 500, 100, 50, 20, 10, 5, 2, 1)

print("Currency Denomination breakdown:")
for note in denominations:
if amount >= note:
count = amount // note
amount = amount % note
print("Notes of", note, "=", count)
```

● OUTPUT

```
Enter total amount of money: 2868
Currency Denomination breakdown:
Notes of 2000 = 1
Notes of 500 = 1
Notes of 100 = 3
Notes of 50 = 1
Notes of 10 = 1
Notes of 5 = 1
Notes of 2 = 1
Notes of 1 = 1
```

Q3. Find Largest Among Three Numbers

◆ PYTHON CODE

```
a = float(input("Enter 1st no: "))
b = float(input("Enter 2nd no: "))
c = float(input("Enter 3rd no: "))

if a >= b and a >= c:
largest = a
elif b >= a and b >= c:
largest = b
else:
largest = c

print("Largest number =", largest)
```

● OUTPUT

```
Enter 1st no: 45
Enter 2nd no: 89
Enter 3rd no: 12
Largest number = 89.0
```

Q4. Calculate Down Payment Based on Credit

◆ PYTHON CODE

```
sales_price = float(input("Enter the sales price of the item: "))
credit_amount = float(input("Enter your credit amount: "))

if credit_amount >= 450000:
down_payment = 0.10 * sales_price
else:
down_payment = 0.20 * sales_price

print("Required Down Payment = ", round(down_payment, 2))
```

● OUTPUT

```
Enter the sales price of the item: 500000
Enter your credit amount: 480000
Required Down Payment = 50000.0
```

Q5. Password Validity Checker

◆ PYTHON CODE

```
password = input("Enter a password to validate: ")

# Track flags for rule checking
has_lower = False
has_upper = False
has_digit = False
has_special = False
special_chars = "$#@"

# Check length limits
if 6 <= len(password) <= 12:
for char in password:
if char.islower():
has_lower = True
elif char.isupper():
has_upper = True
elif char.isdigit():
has_digit = True
elif char in special_chars:
has_special = True

# Validate if all criteria are satisfied
if has_lower and has_upper and has_digit and has_special:
print("Password Status: Valid Password")
else:
print("Password Status: Invalid Password (Missing required character types)")
else:
print("Password Status: Invalid Password (Length must be between 6 and 12 characters)")
```

● OUTPUT

```
Enter a password to validate: Secr3t@12
Password Status: Valid Password
```

Assignment 5

Q1. Check if a Number is an Armstrong Number

◆ PYTHON CODE

```
num = int(input("Enter a number: "))

# Store the number in a temporary variable and calculate total digits
temp = num
num_str = str(num)
n = len(num_str)
sum_of_powers = 0

# Extract digits and raise them to the power of total digits
while temp > 0:
digit = temp % 10
sum_of_powers += digit ** n
temp //= 10

if num == sum_of_powers:
print(num, "is an Armstrong number.")
else:
print(num, "is not an Armstrong number.")
```

● OUTPUT

```
Enter a number: 153
153 is an Armstrong number.
```

Q2. Display Fibonacci Series up to N Terms

◆ PYTHON CODE

```
terms = int(input("Enter the number of terms: "))

n1, n2 = 0, 1
count = 0

if terms <= 0:
print("Please enter a positive integer.")
elif terms == 1:
print("Fibonacci series up to", terms, ":", n1)
else:
print("Fibonacci series:")
while count < terms:
print(n1, end=" ")
nth = n1 + n2
# Update values
n1 = n2
n2 = nth
count += 1
print() # For new line
```

● OUTPUT

```
Enter the number of terms: 5
Fibonacci series:
0 1 1 2 3
```

Q3. Calculate GCD and LCM of Two Numbers

◆ PYTHON CODE

```
a = int(input("Enter 1st no: "))
b = int(input("Enter 2nd no: "))

# Store original values to calculate LCM later
num1, num2 = a, b

# Calculate GCD using the Euclidean algorithm
while b != 0:
temp = b
b = a % b
a = temp

gcd = a
lcm = (num1 * num2) // gcd

print("GCD =", gcd)
print("LCM =", lcm)
```

● OUTPUT

```
Enter 1st no: 12
Enter 2nd no: 18
GCD = 6
LCM = 36
```

Q4. Find the Largest Number and Average in a Series

◆ PYTHON CODE

```
n = int(input("How many numbers do you want to enter? "))

if n <= 0:
print("Invalid amount.")
else:
# Initialize values with the first element
first_num = float(input("Enter element 1: "))
largest = first_num
total_sum = first_num

for i in range(2, n + 1):
num = float(input("Enter element " + str(i) + ": "))
total_sum += num
if num > largest:
largest = num

average = total_sum / n
print("Largest number =", largest)
print("Average of the series =", round(average, 2))
```

● OUTPUT

```
How many numbers do you want to enter? 4
Enter element 1: 10
Enter element 2: 45
Enter element 3: 23
Enter element 4: 12
Largest number = 45.0
Average of the series = 22.5
```

Q5. Check if a Number is a Palindrome

◆ PYTHON CODE

```
num = int(input("Enter a number: "))
temp = num
reverse = 0

while temp > 0:
digit = temp % 10
reverse = (reverse * 10) + digit
temp //= 10

if num == reverse:
print(num, "is a Palindrome.")
else:
print(num, "is not a Palindrome.")
```

● OUTPUT

```
Enter a number: 1221
1221 is a Palindrome.
```

Q6. Find All Prime Numbers Within a Range

◆ PYTHON CODE

```
lower = int(input("Enter lower range: "))
upper = int(input("Enter upper range: "))

print("Prime numbers between", lower, "and", upper, "are:")

for num in range(lower, upper + 1):
if num > 1:
is_prime = True
for i in range(2, int(num ** 0.5) + 1):
if num % i == 0:
is_prime = False
break
if is_prime:
print(num, end=" ")
print() # For new line
```

● OUTPUT

```
Enter lower range: 10
Enter upper range: 30
Prime numbers between 10 and 30 are:
11 13 17 19 23 29
```

Q7. Print the Asterisk Triangle Pattern

◆ PYTHON CODE

```
lines = int(input("Enter number of lines: "))

for i in range(1, lines + 1):
for j in range(i):
print("*", end=" ")
print()
```

● OUTPUT

```
Enter number of lines: 5
* * * * * * * * * * * * * * * ```

---

## 8. Search for a Value in a List Using a Loop

### Python Code
```python
# Sample list creation
numbers = [10, 25, 33, 47, 50, 89, 92]
print("Given List:", numbers)

search_value = int(input("Enter the value to search: "))

found = False
index = -1

for i in range(len(numbers)):
if numbers[i] == search_value:
found = True
index = i
break

if found:
print("Value", search_value, "found at index position", index)
else:
print("Value", search_value, "not found in the list.")
Output
Plaintext
Given List: [10, 25, 33, 47, 50, 89, 92]
Enter the value to search: 47
Value 47 found at index position 3
```

Assignment 6
```

Q1. Functionality of Break, Continue, and Pass

◆ PYTHON CODE

```
sentence = "Haldia Institute of Technology"

print("--- Testing BREAK (stops at 's' or 't') ---")
for char in sentence:
if char == 's' or char == 't':
break
print(char, end="")
print("\n")

print("--- Testing CONTINUE (skips 's' and 't') ---")
for char in sentence:
if char == 's' or char == 't':
continue
print(char, end="")
print("\n")

print("--- Testing PASS (does nothing, prints everything) ---")
for char in sentence:
if char == 's' or char == 't':
pass
print(char, end="")
print()
```

● OUTPUT

```
--- Testing BREAK (stops at 's' or 't') ---
Haldia In

--- Testing CONTINUE (skips 's' and 't') ---
Haldia Initue of Echnology

--- Testing PASS (does nothing, prints everything) ---
Haldia Institute of Technology
```

Q2. Access Characters in Forward and Backward Direction

◆ PYTHON CODE

```
text = input("Enter a string: ")
n = len(text)

print("\nForward Direction:")
for i in range(n):
print(text[i], end=" ")
print()

print("\nBackward Direction:")
for i in range(n - 1, -1, -1):
print(text[i], end=" ")
print()
```

● OUTPUT

```
Enter a string: HALDIA

Forward Direction:
H A L D I A

Backward Direction:
A I D L A H
```

Q3. Display All Positions of Substring in a Main String

◆ PYTHON CODE

```
main_str = input("Enter main string: ")
sub_str = input("Enter substring to find: ")

print("Finding all positions...")
flag = False
pos = -1

while True:
# Find next occurrence starting from index (pos + 1)
pos = main_str.find(sub_str, pos + 1)
if pos == -1:
break
print("Found at index position:", pos)
flag = True

if not flag:
print("Substring not found in the main string.")
```

● OUTPUT

```
Enter main string: abcdabcdabcd
Enter substring to find: cd
Finding all positions...
Found at index position: 2
Found at index position: 6
Found at index position: 10
```

Q4. Print Characters at Odd and Even Positions

◆ PYTHON CODE

```
text = input("Enter a string: ")

print("Characters at Even indexes (0, 2, 4...):")
for i in range(0, len(text), 2):
print(text[i], end=" ")
print()

print("Characters at Odd indexes (1, 3, 5...):")
for i in range(1, len(text), 2):
print(text[i], end=" ")
print()
```

● OUTPUT

```
Enter a string: PYTHON
Characters at Even indexes (0, 2, 4...):
P T O
Characters at Odd indexes (1, 3, 5...):
Y H N
```

Q5. Remove Duplicate Characters of a Given String

◆ PYTHON CODE

```
text = input("Enter a string: ")
result = ""

for char in text:
if char not in result:
result += char

print("String after removing duplicates:", result)
```

● OUTPUT

```
Enter a string: engineering
String after removing duplicates: egni r
```

Q6. Merge Two Strings Into One

◆ PYTHON CODE

```
str1 = input("Enter 1st string: ")
str2 = input("Enter 2nd string: ")

# Simple concatenation using the '+' operator
merged_str = str1 + str2

print("Merged String =", merged_str)
```

● OUTPUT

```
Enter 1st string: Hello
Enter 2nd string: World
Merged String = Hello World
```

Assignment 7

Q1. Traverse and Display All Even Numbers in a List

◆ PYTHON CODE

```
# Sample input list
numbers = [12, 35, 22, 9, 44, 73, 80, 15]
print("Original List:", numbers)

print("Even numbers present in the list:")
for num in numbers:
if num % 2 == 0:
print(num, end=" ")
print()
```

● OUTPUT

```
Original List: [12, 35, 22, 9, 44, 73, 80, 15]
Even numbers present in the list:
12 22 44 80
```

Q2. Check and Count Vowels Present in a Word

◆ PYTHON CODE

```
word = input("Enter a word: ")
vowels_list = ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U']

found_vowels = []
count = 0

for char in word:
if char in vowels_list:
found_vowels.append(char)
count += 1

print("Vowels found in the word:", found_vowels)
print("Total count of vowels =", count)
```

● OUTPUT

```
Enter a word: Education
Vowels found in the word: ['E', 'd', 'u', 'c', 'a', 't', 'i', 'o', 'n']
Total count of vowels = 5
```

Q3. Find Largest and Smallest Number Present in a List

◆ PYTHON CODE

```
numbers = [45, 12, 89, 5, 67, 23]
print("Original List:", numbers)

# --- Method 1: Using Built-in Methods ---
largest_builtin = max(numbers)
smallest_builtin = min(numbers)

# --- Method 2: Without Using Built-in Methods ---
largest_custom = numbers[0]
smallest_custom = numbers[0]

for num in numbers:
if num > largest_custom:
largest_custom = num
if num < smallest_custom:
smallest_custom = num

print("\n[Using Built-in Methods]")
print("Largest =", largest_builtin, "| Smallest =", smallest_builtin)

print("\n[Without Using Built-in Methods]")
print("Largest =", largest_custom, "| Smallest =", smallest_custom)
```

● OUTPUT

```
Original List: [45, 12, 89, 5, 67, 23]

[Using Built-in Methods]
Largest = 89 | Smallest = 5

[Without Using Built-in Methods]
Largest = 89 | Smallest = 5
```

Q4. Remove Duplicate Values Present in a List

◆ PYTHON CODE

```
items = [10, 20, 10, 30, 40, 20, 50, 30]
print("Original List:", items)

unique_items = []
for val in items:
if val not in unique_items:
unique_items.append(val)

print("List after removing duplicates:", unique_items)
```

● OUTPUT

```
Original List: [10, 20, 10, 30, 40, 20, 50, 30]
List after removing duplicates: [10, 20, 30, 40, 50]
```

Q5. Reverse a List

◆ PYTHON CODE

```
list1 = [1, 2, 3, 4, 5]
list2 = [1, 2, 3, 4, 5]
print("Original List:", list1)

# --- Method 1: Using Built-in Method ---
list1.reverse() # Reverses in-place

# --- Method 2: Without Using Built-in Method ---
reversed_custom = []
for i in range(len(list2) - 1, -1, -1):
reversed_custom.append(list2[i])

print("\n[Using Built-in Method reverse()]")
print("Reversed List:", list1)

print("\n[Without Using Built-in Method]")
print("Reversed List:", reversed_custom)
```

● OUTPUT

```
Original List: [1, 2, 3, 4, 5]

[Using Built-in Method reverse()]
Reversed List: [5, 4, 3, 2, 1]

[Without Using Built-in Method]
Reversed List: [5, 4, 3, 2, 1]
```

Q6. Check if Elements Are in Ascending Order or Not

◆ PYTHON CODE

```
# User can switch comments below to test both scenarios
nums = [10, 25, 30, 45, 60]
# nums = [10, 45, 25, 30, 60]

print("Given List:", nums)

is_ascending = True

for i in range(len(nums) - 1):
if nums[i] > nums[i + 1]:
is_ascending = False
break

if is_ascending:
print("Result: The list elements are in ascending order.")
else:
print("Result: The list elements are NOT in ascending order.")
```

● OUTPUT

```
Given List: [10, 25, 30, 45, 60]
Result: The list elements are in ascending order.
```

Assignment 8

Q1. Sum and Average of Elements in an Integer List

◆ PYTHON CODE

```
numbers = [10, 20, 30, 40, 50]
print("Given List:", numbers)

total_sum = 0
for num in numbers:
total_sum += num

average = total_sum / len(numbers)

print("Sum of elements =", total_sum)
print("Average of elements =", average)
```

● OUTPUT

```
Given List: [10, 20, 30, 40, 50]
Sum of elements = 150
Average of elements = 30.0
```

Q2. Sort Elements of a List Without Using Built-in Functions

◆ PYTHON CODE

```
nums = [64, 34, 25, 12, 22, 11, 90]
print("Original List:", nums)

n = len(nums)
# Implementing Bubble Sort Algorithm
for i in range(n):
for j in range(0, n - i - 1):
if nums[j] > nums[j + 1]:
# Swap elements
nums[j], nums[j + 1] = nums[j + 1], nums[j]

print("Sorted List:", nums)
```

● OUTPUT

```
Original List: [64, 34, 25, 12, 22, 11, 90]
Sorted List: [11, 12, 22, 25, 34, 64, 90]
```

Q3. Find Elements Present Odd Number of Times in a List

◆ PYTHON CODE

```
items = [2, 3, 5, 4, 5, 2, 4, 3, 5, 2, 4, 4, 2]
print("Given List:", items)

odd_occurring_elements = []

for val in items:
# Count frequency manually without using list.count()
count = 0
for element in items:
if element == val:
count += 1

if count % 2 != 0 and val not in odd_occurring_elements:
odd_occurring_elements.append(val)

print("Elements present an odd number of times:", odd_occurring_elements)
```

● OUTPUT

```
Given List: [2, 3, 5, 4, 5, 2, 4, 3, 5, 2, 4, 4, 2]
Elements present an odd number of times: [5]
```

Q4. Merge Two Lists and Find the Even Numbers

◆ PYTHON CODE

```
list1 = [11, 22, 33]
list2 = [44, 55, 66]
print("List 1:", list1)
print("List 2:", list2)

# Merge the two lists
merged_list = list1 + list2
print("Merged List:", merged_list)

print("Even numbers in the merged list:")
for num in merged_list:
if num % 2 == 0:
print(num, end=" ")
print()
```

● OUTPUT

```
List 1: [11, 22, 33]
List 2: [44, 55, 66]
Merged List: [11, 22, 33, 44, 55, 66]
Even numbers in the merged list:
22 44 66
```

Q5. Tuple Concatenation and Operations

◆ PYTHON CODE

```
tup1 = (12, 45, 78)
tup2 = (23, 56, 89)
print("Tuple 1:", tup1)
print("Tuple 2:", tup2)

# Concatenate tuples
new_tup = tup1 + tup2
print("Concatenated Tuple:", new_tup)

# a) Find length
print("a) Length of tuple =", len(new_tup))

# b) Find max and min
print("b) Maximum element =", max(new_tup), "| Minimum element =", min(new_tup))

# c) Search an element
search_item = int(input("c) Enter element to search: "))
if search_item in new_tup:
print(" ", search_item, "is present in the tuple.")
else:
print(" ", search_item, "is not present in the tuple.")

# d) Delete the tuple
del new_tup
print("d) Concatenated tuple successfully deleted from memory.")
```

● OUTPUT

```
Tuple 1: (12, 45, 78)
Tuple 2: (23, 56, 89)
Concatenated Tuple: (12, 45, 78, 23, 56, 89)
a) Length of tuple = 6
b) Maximum element = 89 | Minimum element = 12
c) Enter element to search: 45
45 is present in the tuple.
d) Concatenated tuple successfully deleted from memory.
```

Q6. Check if a Tuple Contains All Unique Elements

◆ PYTHON CODE

```
# Sample testing tuples
tup_a = (10, 20, 30, 40, 50)
# tup_a = (10, 20, 30, 20, 50)

print("Given Tuple:", tup_a)

is_unique = True
seen_items = []

for item in tup_a:
if item in seen_items:
is_unique = False
break
seen_items.append(item)

if is_unique:
print("Result: The tuple contains all unique elements.")
else:
print("Result: The tuple contains duplicate elements.")
```

● OUTPUT

```
Given Tuple: (10, 20, 30, 40, 50)
Result: The tuple contains all unique elements.
```

Assignment 9

Q1. Enter Name and Percentage Marks in a Dictionary

◆ PYTHON CODE

```
n = int(input("Enter number of students: "))
student_records = {}

for i in range(n):
name = input("Enter student name: ")
percentage = float(input("Enter percentage marks: "))
student_records[name] = percentage

print("\n--- Student Information ---")
for name, percentage in student_records.items():
print("Name:", name, "| Percentage:", percentage, "%")
```

● OUTPUT

```
Enter number of students: 3
Enter student name: Amit
Enter percentage marks: 85.5
Enter student name: Priya
Enter percentage marks: 92.0
Enter student name: Rahul
Enter percentage marks: 78.4

--- Student Information ---
Name: Amit | Percentage: 85.5 %
Name: Priya | Percentage: 92.0 %
Name: Rahul | Percentage: 78.4 %
```

Q2. Take Dictionary from Keyboard and Print Sum of Values

◆ PYTHON CODE

```
n = int(input("Enter number of items in dictionary: "))
my_dict = {}

for i in range(n):
key = input("Enter key: ")
val = float(input("Enter numeric value: "))
my_dict[key] = val

print("Given Dictionary:", my_dict)

total_sum = 0
for value in my_dict.values():
total_sum += value

print("Sum of all values =", total_sum)
```

● OUTPUT

```
Enter number of items in dictionary: 3
Enter key: item1
Enter numeric value: 150
Enter key: item2
Enter numeric value: 250
Enter key: item3
Enter numeric value: 100
Given Dictionary: {'item1': 150.0, 'item2': 250.0, 'item3': 100.0}
Sum of all values = 500.0
```

Q3. Count Number of Occurrences of Each Letter in a String

◆ PYTHON CODE

```
text = input("Enter a string: ")
letter_counts = {}

for char in text:
# Optional: ignore spaces if you only want letters
if char != " ":
if char in letter_counts:
letter_counts[char] += 1
else:
letter_counts[char] = 1

print("Letter frequencies:")
for letter, count in letter_counts.items():
print(letter, "=", count)
```

● OUTPUT

```
Enter a string: success
Letter frequencies:
s = 3
u = 1
c = 2
e = 1
```

Q4. Count Number of Occurrences of Each Vowel in a String

◆ PYTHON CODE

```
text = input("Enter a string: ")

# Initialize dictionary with all vowels set to 0
vowel_counts = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}

# Convert string to lower case to count both upper and lower case vowels
for char in text.lower():
if char in vowel_counts:
vowel_counts[char] += 1

print("Vowel frequencies:")
for vowel, count in vowel_counts.items():
print(vowel, "=", count)
```

● OUTPUT

```
Enter a string: Haldia Institute
Vrequencies of vowels:
a = 2
e = 1
i = 3
o = 0
u = 1
```

Q5. Search Student Marks by Name using Dictionary

◆ PYTHON CODE

```
n = int(input("Enter number of students to add: "))
gradebook = {}

for i in range(n):
name = input("Enter student name: ")
marks = int(input("Enter marks: "))
gradebook[name] = marks

print("\n--- Gradebook Created ---")
search_name = input("Enter student name to find marks: ")

if search_name in gradebook:
print(search_name, "scored:", gradebook[search_name], "marks")
else:
print("Student record not found.")
```

● OUTPUT

```
Enter number of students to add: 2
Enter student name: Rohan
Enter marks: 88
Enter student name: Sneha
Enter marks: 95

--- Gradebook Created ---
Enter student name to find marks: Sneha
Sneha scored: 95 marks
```

Q6. Dictionary Operations (Access, Key Count, Update)

◆ PYTHON CODE

```
# Create initial dictionary
profile = {
'name': 'John Doe',
'age': 21,
'address': 'West Bengal',
'type': 'Student'
}
print("Initial Dictionary:", profile)

# Display total keys present
print("Total keys present =", len(profile))

# Display value of the key 'name'
print("Value of key 'name' =", profile['name'])

# Update dictionary with two more keys
profile['basic'] = 25000
profile['nature'] = 'Permanent'

print("\nUpdated Dictionary:")
print(profile)
```

● OUTPUT

```
Initial Dictionary: {'name': 'John Doe', 'age': 21, 'address': 'West Bengal', 'type':
'Student'}
Total keys present = 4
Value of key 'name' = John Doe

Updated Dictionary:
{'name': 'John Doe', 'age': 21, 'address': 'West Bengal', 'type': 'Student', 'basic': 25000,
'nature': 'Permanent'}
```

Assignment 10

Q1. Employee Greeting and DA Calculation Function

◆ PYTHON CODE

```
def employee_details(first_name, last_name, basic_salary):
# Greeting message
print("Welcome,", first_name, last_name + "!")

# DA calculation (50% of basic)
da = 0.50 * basic_salary
print("Dearness Allowance (DA) = ", da)

# Input data from user
f_name = input("Enter first name: ")
l_name = input("Enter last name: ")
salary = float(input("Enter basic salary: "))

# Function call
employee_details(f_name, l_name, salary)
```

● OUTPUT

```
Enter first name: Rajesh
Enter last name: Kumar
Enter basic salary: 45000
Welcome, Rajesh Kumar!
Dearness Allowance (DA) = 22500.0
```

Q2. Numbers Divisible by 7 but Not 5 Within a Range

◆ PYTHON CODE

```
def find_numbers(lower, upper):
print("Numbers divisible by 7 but not 5 between", lower, "and", upper, "are:")
for num in range(lower, upper + 1):
if num % 7 == 0 and num % 5 != 0:
print(num, end=" ")
print() # For new line

# Input parameters from user
start = int(input("Enter lower limit: "))
end = int(input("Enter upper limit: "))

# Function call
find_numbers(start, end)
```

● OUTPUT

```
Enter lower limit: 1
Enter upper limit: 50
Numbers divisible by 7 but not 5 between 1 and 50 are:
7 14 21 28 42 49
```

Q3. Find the Largest Among Three Numbers Using a Function

◆ PYTHON CODE

```
def find_largest(num1, num2, num3):
if num1 >= num2 and num1 >= num3:
return num1
elif num2 >= num1 and num2 >= num3:
return num2
else:
return num3

# Input numbers from user
a = float(input("Enter 1st no: "))
b = float(input("Enter 2nd no: "))
c = float(input("Enter 3rd no: "))

# Function call
largest = find_largest(a, b, c)
print("The largest number is =", largest)
```

● OUTPUT

```
Enter 1st no: 34.5
Enter 2nd no: 89.2
Enter 3rd no: 12.1
The largest number is = 89.2
```

Q4. Reverse a String Using a Function

◆ PYTHON CODE

```
def reverse_string(text):
reversed_str = ""
# Loop backward through the string index
for i in range(len(text) - 1, -1, -1):
reversed_str += text[i]
return reversed_str

# Input string from user
user_str = input("Enter a string: ")

# Function call
result = reverse_string(user_str)
print("Reversed String =", result)
```

● OUTPUT

```
Enter a string: HALDIA
Reversed String = AIDLAH
```